

Measuring

We have already measured application startup time in the previous lab.

Remove unnecessary functionality

Compiling ffmpeg with a reduced configuration

In our system, we use a generic version of that was built with support for too many codecs and options that we actually do not need in our very special case.

So, let's try to find out what the minimum requirements for are.

A first thing to do is to look at the ffmpeg logs:

```
Input #0, video4linux2,v4l2, from '/dev/video0':
  Duration: N/A, start: 93.369296, bitrate: N/A
    Stream #0:0: Video: mjpeg (Baseline), yuvj422p(pc, bt470bg/unknown/unknown),
    544x288, 30 fps, 30 tbr, 1000k tbn, 1000k tbc
Stream mapping:
  Stream #0:0 -> #0:0 (mjpeg (native) -> rawvideo (native))
Press [q] to stopable to decode APP fields: Invalid data found when processing input
[swscaler @ 0x80f50] deprecated pixel format used, make sure you did set range
correctly
[swscaler @ 0x80f50] No accelerated colorspace conversion found from yuv422p to
rgb565le.
Output #0, fbdev, to '/dev/fb0':
  Metadata:
    encoder           : Lavf58.29.100
    Stream #0:0: Video: rawvideo (RGB[16] / 0x10424752), rgb565le, 544x288, q=2-31,
    75202 kb/s, 30 fps, 30 tbn, 30 tbc
    Metadata:
      encoder         : Lavc58.54.100 rawvideo
```

Here we see that is using:

- Input from a device, decoding an stream.
- Encoding a stream, written to an output device.
- A software scaler to resize the input video for our LCD screen

Let's check 's script, and see what its options are:

```
cd $HOME/__$SESSION_NAME__-labs/rootfs/buildroot/output/build/ffmpeg-4.4.4 ./configure --help
\end{bashinput}
```

We see that `configure` has precisely three interesting options: `--list-encoders`, `--list-decoders`, `--list-filters`, `--list-outdevs` and `--list-indevs`.

Run `configure` with each of those and recognize the features that we need to enable.

Following these findings, here's how we are going to modify Buildroot's configuration for `ffmpeg`.

```
\begin{bashinput} cd$HOME/__$SESSION_NAME__-labs/rootfs/buildroot/ make menuconfig
```

In Buildroot's configuration interface, in options:

- Set to
- Set to

- Empty the , , , and settings.
- Set to
- For and , individual device selection is not possible, so we will configure devices manually in the next field. So, empty such settings.
- Set to

Now, let's get Buildroot to recompile , taking our new settings into account:

```
make ffmpeg-dirclean
make
```

You can now fill the spreadsheet, reusing data from the previous lab:

	A	B	C
1		Total rootfs size	/usr/bin/ffmpeg size
2	Initial configuration		
3	Reduced configuration		
4	Difference		

Do you expect to see differences in execution time, with a reduced configuration? Run the measures with again, and compare with what you got during the previous lab.

If the results surprise you, don't hesitate to show them to your instructor ask for her/his opinion.

Trying to remove further features

Looking at the log which displays enabled configuration settings, try to find further configuration switches which can be removed without breaking the player in our particular system.

Further analysis of the application

With a build system like Buildroot, it's easy to add performance analysis and debugging utilities.

Configure Buildroot to add to your root filesystem. You will find the corresponding configuration option in and then in .

Run Buildroot and reflash your device as usual.

Tracing and profiling with strace

With 's help, you can already have a pretty good understanding of how your application spends its time. You can see all the system calls that it makes and knowing the application, you can guess in which part of the code it is at a given time.

You can also spot unnecessary attempts to open files that do not exist, multiple accesses to the same file, or more generally things that the program was not supposed to do. All these correspond to opportunities to fix and optimize your application.

Once the board has booted, run on the video player application:

```
strace -tt -f -o /tmp/strace.log ffmpeg -f video4linux2 -video_size 544x288 \
-input_format mjpeg -i /dev/video0 -pix_fmt rgb565le -f fbdev /dev/fb0
```

Also have generate a summary:

```
strace -c -f -o /tmp/strace-summary.log ffmpeg ...
```

Take some time to read¹, and see everything that the program is doing. Don't hesitate to lookup the ioctl codes on the Internet to have an idea about what's going on between the player, the camera and the display.

Also have a look at . You will find the number of errors trying to open files that do not exist, and where most time is spent, for example. You can also count the number of memory allocations (using the system call).

Optimizing necessary functionality

At this stage, there is nothing more we can really do to further optimize , unless we are ready to dig into the code and make changes.

However, if the player was your own application, I'm sure this would help to understand how it's actually behaving and how to improve it to make it even faster and smaller.

Putting things back together

Now that we have analyzed the execution of the video player, let's restore the normal configuration for the system:

- Remove support for
- Restore the patch, replacing the most recently applied patch.
- Restoring the automatic execution of in .

As explained in the Buildroot manual², you need to make a full rebuild after disabling packages (such as in our case). Otherwise, such packages will still be present in the filesystem image. Fortunately, full rebuilds are now fast with Buildroot when it's using a prebuilt toolchain:

```
make clean
make
```

Update your root filesystem and then reboot your system through , copying the output to .

Run the experiment 3 times and fill the spreadsheet:

	A	B	C	D	E	F	G	H
1		Run 1	Run 2	Run 3	Average	Std deviation	Delta with previous	Time spent in...
2	"Run /sbin/init" timestamp							
3	"Starting ffmpeg" timestamp							← Init scripts
4	"First frame decoded" timestamp							← Application

¹At this stage, when you have to open files directly on the board, some familiarity with the basic commands of the editor becomes useful. See https://bootlin.com/doc/command_memento.pdf for a basic command summary. Otherwise, you can use the more rudimentary command. You can also copy the files to your PC, using a USB drive, for example.

²<https://buildroot.org/downloads/manual/manual.html#full-rebuild>